LAPORAN

UJIAN TEGAH SEMESTER

4143108 – SOFTWARE ARCHITECTURE AND DESIGN

Disusun Oleh :

NIM	NAMA
11423018	Eduward Gilbert Simanjuntak

SARJANA TERAPAN TEKNOLOGI REKAYASA PERANGKAT LUNAK (TRPL)

INSTITUT TEKNOLOGI DEL

2026

Daftar ISI

Case Study 1: MediTrack A Digital Healthcare Platform 3

1. Soal ini dikerjakan pada branch case_study1. 3

2. Design Thinking & Architecture Selection 3

1. Separation of Concerns (SoC) / Bounded Context 3

3. System Decomposition & Modeling 6

4. [Architecture Visualization] As a leader, all your team must be understand the solusion you provide, Draw a high-level architecture diagram of your proposed solution. 11

5. Implementasi (Source Code Overview) 12

6. [Bonus Task] Identify potential performance bottlenecks in your architecture and propose mitigation strategies! 14

Case Study 2: MediTrack Transformation 16

1. Soal ini dikerjakan pada branch case_study2 16

2. To simplify the main concept and understanding, please provide the use case based on the case study! 16

3. Architecture Diagram 19

4. Reasoning and Trade-offs 21

5. Migration Strategy 22

6. [Implementasi] Monolith Representation (Simulation) 22

7. Microservices Mapping 27

Case Study 1: MediTrack A Digital Healthcare Platform

1. **Soal ini dikerjakan pada branch case_study1.**
2. **Design Thinking & Architecture Selection**

Dalam pengembangan platform MediTrack, dipilih pendekatan Monolithic Architecture dengan konsep Modular Monolith menggunakan framework Laravel (PHP). Pendekatan ini dipilih untuk menyeimbangkan kebutuhan pengembangan cepat, kemudahan pemeliharaan, serta kesiapanF terhadap pertumbuhan sistem di masa depan.

a. Justify the choice with at least three architectural principles

1. Separation of Concerns (SoC) / Bounded Context

Prinsip Separation of Concerns diterapkan melalui pendekatan Domain-Driven Design (DDD), di mana setiap domain utama dalam sistem dipisahkan ke dalam bounded context yang jelas, seperti User Management, Appointment Scheduling, Electronic Health Records (EHR), Pharmacy Integration, Payment, dan Analytics. Dengan pemisahan ini, setiap domain memiliki tanggung jawab yang spesifik dan tidak saling tumpang tindih. Hal ini sangat penting dalam sistem healthcare karena setiap domain memiliki aturan bisnis, tingkat kompleksitas, dan kebutuhan keamanan yang berbeda. Misalnya, modul EHR memiliki kebutuhan keamanan dan audit yang lebih ketat dibandingkan modul appointment. Dengan SoC, sistem menjadi lebih terstruktur, mudah dipahami, dan lebih mudah dikembangkan tanpa memengaruhi domain lain.

2. Scalability (Horizontal Scaling)

Arsitektur hybrid memungkinkan penerapan horizontal scaling secara selektif pada domain yang membutuhkan. Tidak semua layanan dalam MediTrack memiliki beban kerja yang sama; misalnya, Appointment Scheduling cenderung memiliki traffic tinggi, sementara Analytics membutuhkan pemrosesan data besar secara batch atau streaming. Dengan memecah domain menjadi microservices, setiap layanan dapat di-scale secara independen sesuai kebutuhan. Hal ini meningkatkan efisiensi penggunaan resource dan menghindari pemborosan yang sering terjadi pada arsitektur monolith, di mana seluruh sistem harus di-scale meskipun hanya satu bagian yang membutuhkan peningkatan kapasitas.

3. Security & Compliance Isolation

Dalam sistem healthcare, keamanan data merupakan aspek kritis karena melibatkan informasi sensitif pasien. Dengan pendekatan domain isolation, setiap layanan seperti EHR dan Payment dapat memiliki mekanisme keamanan dan compliance yang disesuaikan dengan kebutuhannya. Misalnya, layanan EHR dapat menerapkan enkripsi data, audit logging, dan access control yang ketat, sementara layanan Payment harus memenuhi standar keamanan transaksi finansial. Pemisahan ini mengurangi risiko kebocoran data dan mempermudah proses audit serta kepatuhan terhadap regulasi. Selain itu, komunikasi antar layanan dapat diamankan menggunakan API Gateway, token-based authentication, dan enkripsi.

b. Identify trade-offs and limitations

Meskipun arsitektur Modular Monolith memiliki berbagai keunggulan, terdapat beberapa trade-off dan keterbatasan yang perlu diperhatikan. Salah satu keterbatasan utama adalah dalam hal skalabilitas, di mana seluruh aplikasi harus diskalakan secara bersamaan, meskipun hanya sebagian fitur yang mengalami peningkatan beban. Hal ini dapat menyebabkan penggunaan sumber daya yang kurang efisien dibandingkan dengan arsitektur microservices yang memungkinkan skalabilitas per layanan.

Aspek	Kelebihan (Trade-off)	Kekurangan/Batasan
Deploy & Setup	Satu deploy, mudah setup, tidak perlu orkestrasi service	Jika satu bagian gagal, seluruh aplikasi bisa terdampak
Kinerja	Komunikasi antar modul sangat cepat (in-process)	Skalabilitas terbatas, sulit scale hanya sebagian modul
Pengembangan	Mudah debug, satu codebase, konsistensi tinggi	Tim besar bisa konflik di codebase yang sama, sulit paralel development
Database	Satu database, integrasi data mudah	Tidak bisa pilih teknologi database berbeda untuk modul berbeda
Deployment	Sederhana, satu pipeline CI/CD	Downtime saat update lebih besar, tidak bisa rolling update per modul
Pengembangan Awal	Cepat untuk MVP/startup	Seiring bertambah besar,

		kompleksitas dan beban maintenance meningkat

c. Suggest how the architecture supports scalability, maintainability, and extensibility

Meskipun memiliki keterbatasan, arsitektur Modular Monolith tetap mampu mendukung kebutuhan skalabilitas, maintainability, dan extensibility jika dirancang dengan tepat.

Aspek	**Dukungan Arsitektur Monolitik Laravel**	**Saran Praktis**
Skalabilitas	- Mudah horizontal scaling (banyak instance, load balancer) - Query & cache terpusat	- Gunakan queue (jobs) untuk proses berat - Optimasi database & cache - Modularisasi service class

Dari sisi skalabilitas, sistem dapat ditingkatkan performanya melalui pendekatan horizontal scaling dengan menambahkan server dan menggunakan load balancer untuk mendistribusikan beban. Selain itu, penggunaan caching seperti Redis dapat membantu mempercepat akses data yang sering digunakan, sementara mekanisme queue memungkinkan pemrosesan tugas secara asynchronous, seperti pengiriman notifikasi dan pemrosesan pembayaran.

Aspek	**Dukungan Arsitektur Monolitik Laravel**	**Saran Praktis**
Maintainability	-Struktur folder jelas (Controller, Service, Model) - Satu codebase, mudah debug	-Pisahkan logika bisnis di service - Tambahkan unit/feature test - Dokumentasi rutin

Dari sisi maintainability, struktur modular yang diterapkan memungkinkan sistem lebih mudah dipelihara karena setiap bagian memiliki batasan yang jelas. Penggunaan pola desain seperti service layer dan repository pattern juga membantu menjaga kebersihan kode serta mempermudah proses debugging dan pengembangan lanjutan. Dengan struktur yang terorganisir, developer dapat dengan mudah memahami dan memperbaiki bagian tertentu

tanpa harus memahami keseluruhan sistem.

Dokumentasi rutin

Aspek	**Dukungan Arsitektur Monolitik Laravel**	**Saran Praktis**
Extensibility	-Struktur folder jelas (Controller, Service, Model) - Satu codebase, mudah debug	-Pisahkan logika bisnis di service - Tambahkan unit/feature test - Dokumentasi rutin

Sementara itu, dalam hal extensibility, arsitektur ini memberikan fleksibilitas untuk menambahkan fitur baru tanpa harus mengubah struktur utama sistem secara signifikan. Fitur seperti telemedicine, integrasi kecerdasan buatan, maupun pengembangan aplikasi mobile dapat ditambahkan sebagai modul baru. Selain itu, pendekatan modular juga memungkinkan sistem untuk berevolusi ke arah arsitektur microservices di masa depan dengan cara memisahkan modul tertentu menjadi layanan independen sesuai kebutuhan.

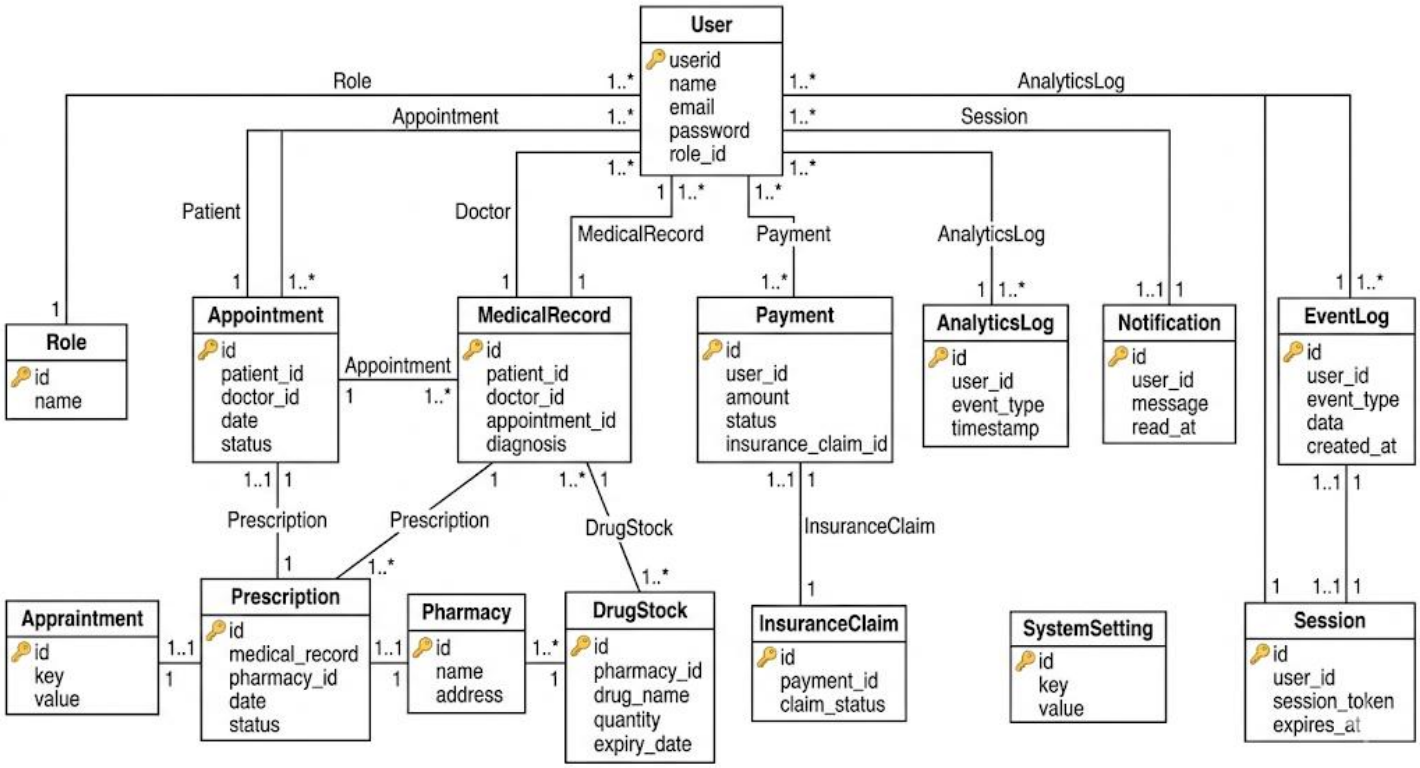
3. System Decomposition & Modeling

a. <u>List the modules/services and their responsibilities.</u>

Modul	**Tanggung Jawab Utama**
User Management	Mengelola data user (pasien, dokter, apoteker, admin), autentikasi, otorisasi, manajemen peran
Appointment	Penjadwalan, pembatalan, riwayat, dan manajemen janji temu antara pasien dan dokter
Medical Record (EHR)	Menyimpan, mengelola, dan menampilkan rekam medis pasien, diagnosis, hasil lab, riwayat pengobatan
Prescription	Membuat, mengelola, dan

	melacak resep obat, integrasi dengan apotek
Pharmacy	Manajemen data apotek, stok obat, pemantauan kadaluarsa, pencarian apotek
Payment	Proses pembayaran, refund, klaim asuransi, pelaporan transaksi
Analytics	Menyajikan dashboard, laporan statistik, analisis performa dokter, tren penggunaan obat, outcome
Authentication	Proses login, logout, registrasi, dan pengelolaan sesi pengguna
Notification/Event	Mengirim notifikasi (email, in-app), event handling (misal: appointment reminder, status update)
Admin Panel	Manajemen sistem, monitoring, pengelolaan data master, pengaturan aplikasi

b. <u>For each module, define key classes/entities and their relationships.</u>



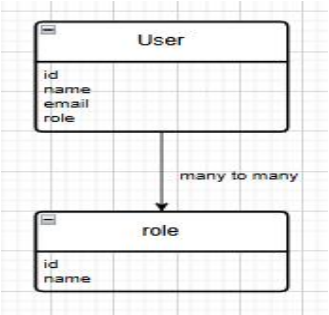
Setiap modul dalam MediTrack memiliki entitas utama yang merepresentasikan objek dalam sistem serta relasi antar entitas tersebut. Pada modul User Management, entitas utama terdiri dari User, Role, dan Profile. Setiap user memiliki satu role, dan setiap user memiliki satu profil yang berisi informasi detail sesuai jenis pengguna. Pada modul Appointment, terdapat entitas Appointment yang berelasi dengan Patient dan Doctor. Satu pasien dapat memiliki banyak appointment, dan satu dokter juga dapat menangani banyak appointment.

Pada modul EHR, entitas utama meliputi MedicalRecord, Prescription, dan LabResult. Setiap medical record dimiliki oleh satu pasien dan dapat memiliki banyak prescription serta hasil lab. Pada modul Pharmacy, terdapat entitas Drug, Inventory, dan Order. Prescription dari modul EHR akan menghasilkan order yang diproses oleh pharmacy. Inventory digunakan untuk mengelola stok obat. Pada modul Payment, entitas seperti Transaction dan InsuranceClaim digunakan untuk mencatat pembayaran yang dilakukan pasien serta klaim yang diajukan ke pihak asuransi. Pada modul Analytics, entitas seperti Report dan Metrics digunakan untuk mengolah dan menyajikan data dari berbagai modul lainnya. Relasi antar modul tetap dijaga seminimal mungkin dengan komunikasi melalui service layer, sehingga tidak terjadi ketergantungan langsung yang berlebihan.

c. Use UML Class Diagrams or component diagrams to illustrate the design.

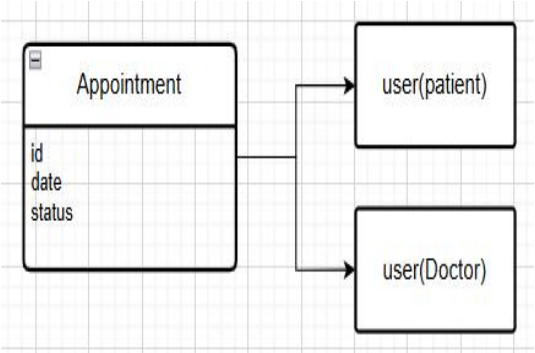
Berikut berapa class diagram yang ada dalam sistem :

- User — Role (Many-to-One)



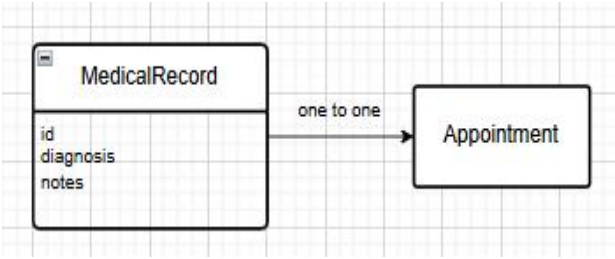
Relasi antara *User* dan *Role* adalah many-to-one, yang berarti banyak pengguna dapat memiliki satu peran yang sama. Dalam sistem MediTrack, setiap user seperti pasien, dokter, atau admin akan memiliki satu role yang menentukan hak aksesnya.

- User — Appointment (Patient & Doctor) (One-to-Many)



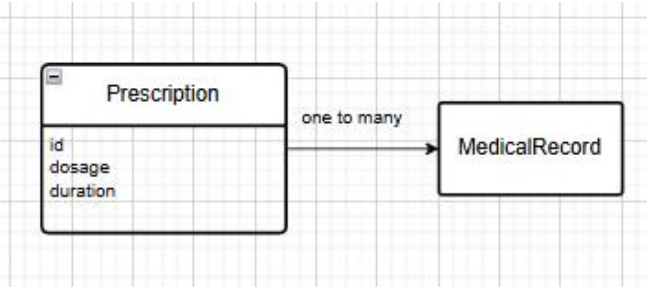
Relasi antara *User* dan *Appointment* terjadi dalam dua konteks, yaitu sebagai pasien dan sebagai dokter. Satu user yang berperan sebagai pasien dapat memiliki banyak appointment, karena pasien dapat melakukan beberapa konsultasi. Demikian pula, satu user sebagai dokter dapat menangani banyak appointment dari berbagai pasien. Setiap appointment hanya terkait dengan satu pasien dan satu dokter.

- Appointment — MedicalRecord (One-to-One)



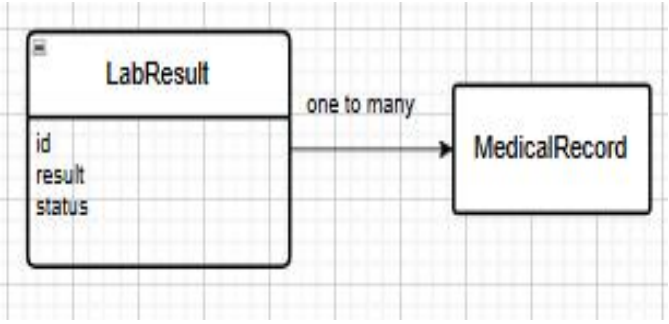
Relasi antara Appointment dan MedicalRecord adalah one-to-one, yang berarti setiap appointment menghasilkan satu rekam medis. Hal ini karena setiap sesi konsultasi akan menghasilkan satu catatan medis yang mencatat diagnosis, tindakan, dan catatan dokter.

- MedicalRecord — Prescription (One-to-Many)



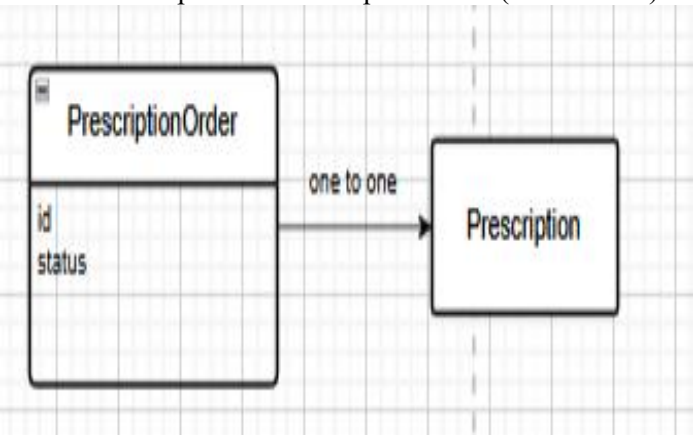
Relasi ini menunjukkan bahwa satu MedicalRecord dapat memiliki banyak Prescription. Dalam satu kunjungan medis, dokter dapat memberikan lebih dari satu resep obat kepada pasien.

- MedicalRecord — LabResult (One-to-Many)



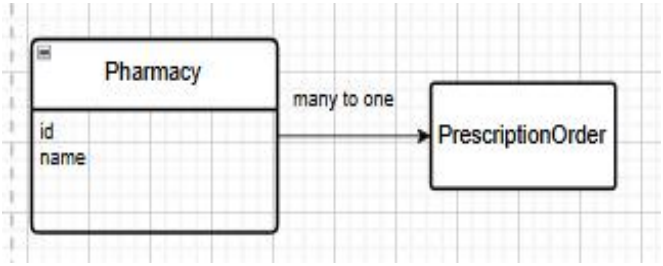
Relasi antara MedicalRecord dan LabResult bersifat one-to-many, di mana satu rekam medis dapat memiliki beberapa hasil laboratorium. Hal ini mencerminkan kondisi nyata bahwa dalam satu pemeriksaan, pasien dapat menjalani beberapa tes laboratorium.

- Prescription — PrescriptionOrder (One-to-One)



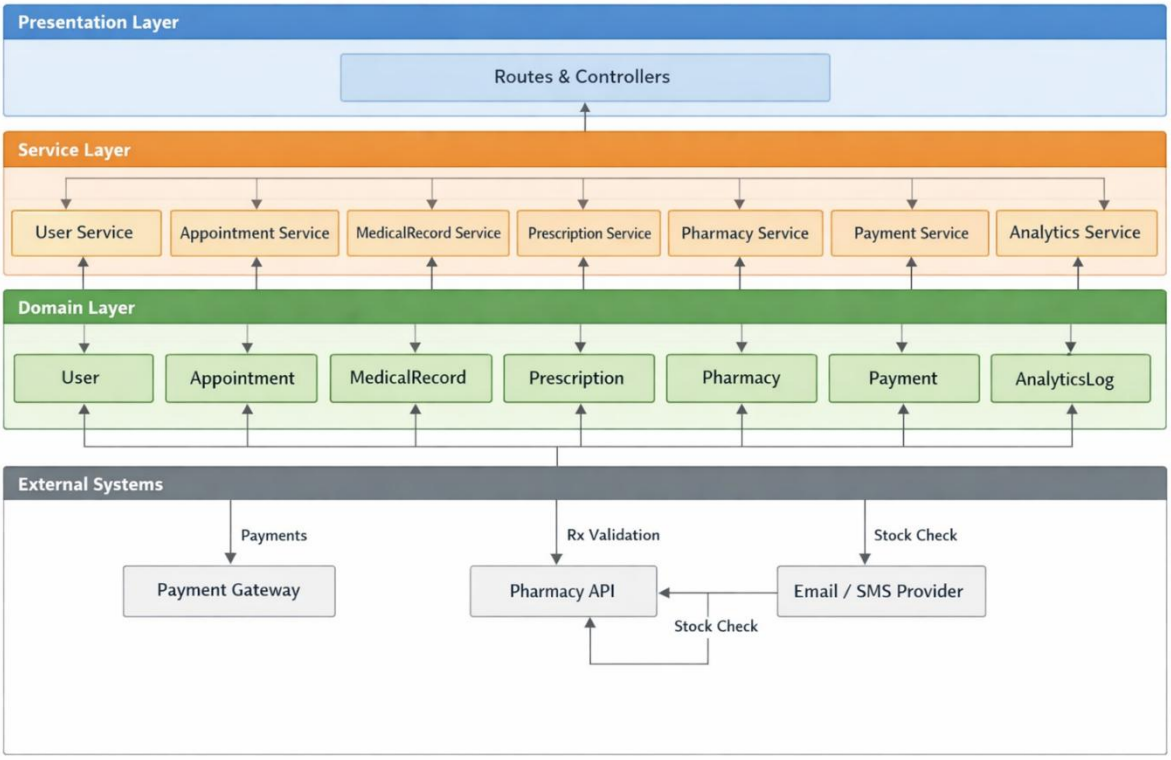
Relasi ini menunjukkan bahwa setiap Prescription akan menghasilkan satu PrescriptionOrder dalam sistem farmasi. Prescription merupakan instruksi medis dari dokter, sedangkan prescription order adalah bentuk implementasinya di apotek. Satu prescription tidak dapat memiliki lebih dari satu order utama dalam konteks sederhana ini.

- PrescriptionOrder — Pharmacy (Many-to-One)



Relasi antara PrescriptionOrder dan Pharmacy adalah many-to-one. Banyak order resep dapat diproses oleh satu pharmacy. Namun, setiap order hanya ditangani oleh satu pharmacy tertentu. Hal ini mencerminkan proses distribusi obat di dunia nyata.

4. **[Architecture Visualization] As a leader, all your team must be understand the solusion you provide, Draw a high-level architecture diagram of your proposed solution.**

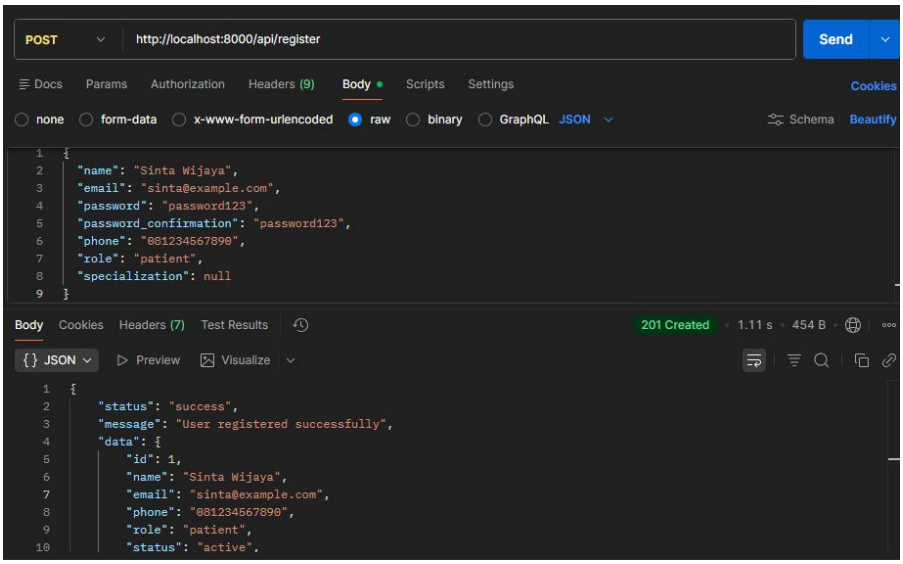


tidak ada terkait dengan eksternal sistem lain

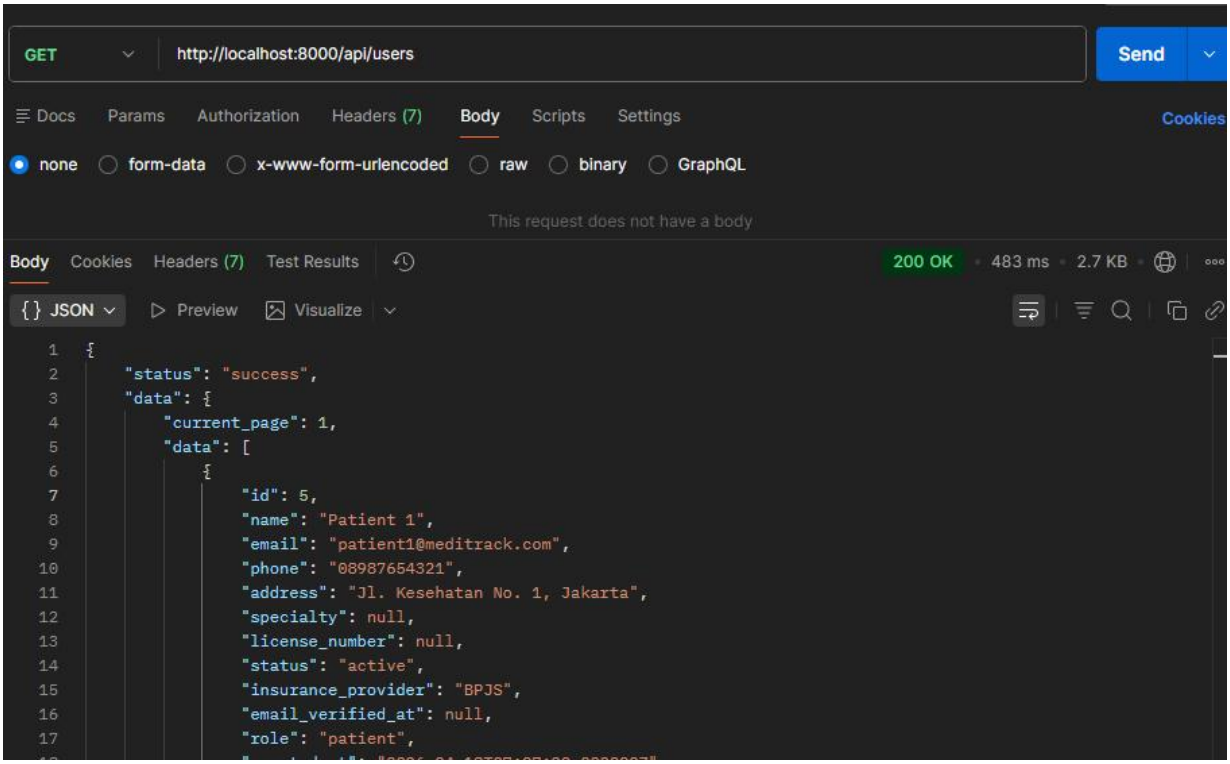
5. **Implementasi (Source Code Overview)**

Uji Endpoint Public

- Daftar Sebagai pasien (POST /api/register)



- Lihat Daftar (User/Pasien/Dokter) GET /api/users



- Lihat Semua Dokter GET /api/users/doctors/all

http://localhost:8000/api/users/doctors/all

GET http://localhost:8000/api/users/doctors/all

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (7) Test Results 200 OK 470 ms 1.18 KB

JSON Preview Visualize


```
{
    "status": "success",
    "data": [
        {
            "id": 2,
            "name": "Dr. Ahmad",
            "email": "doctor1@meditrack.com",
            "phone": "08123456781",
            "address": null,
            "specialty": "Umum",
            "license_number": "LIC-DOC-0001",
            "status": "active",
            "insurance_provider": null,
            "email_verified_at": null,
            "role": "doctor",
            "created_at": "2026-04-18T07:07:21.000000Z",
            "updated_at": "2026-04-18T07:07:21.000000Z"
```


- Login Sebagai Pasien

UTS APPL / http://localhost:8000/api/register

POST http://localhost:8000/api/login

Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

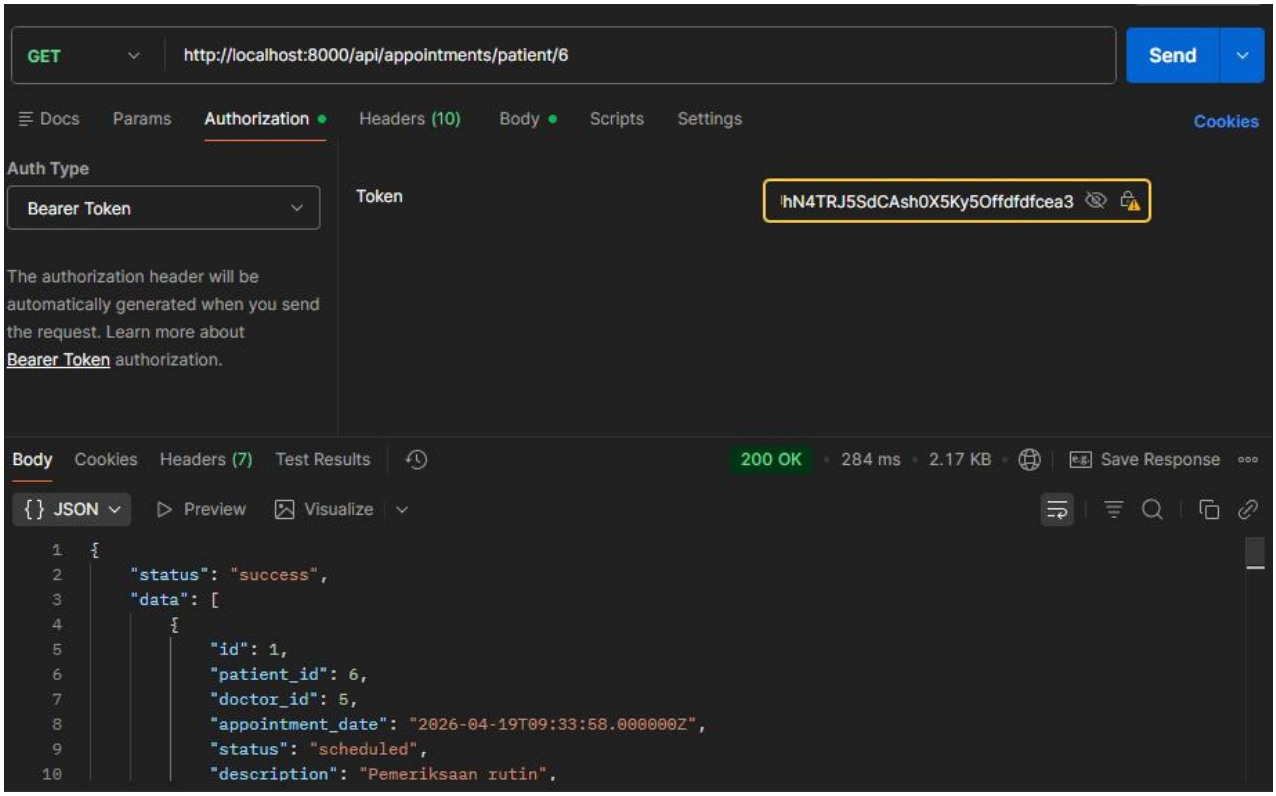

```
{
  "email": "sinta@example.com",
  "password": "password123"
}
```


Body Cookies Headers (7) Test Results 200 OK 1.11 s 464 B Save Response

JSON Preview Visualize


```
{
    "status": "success",
    "message": "Login successful",
    "data": {
        "user": {
            "id": 1,
            "name": "Sinta Wijaya",
            "email": "sinta@example.com",
            "phone": "081234567890",
            "role": "patient",
```

- Lihat janji temu pasien GET /api/appointments/patient/6



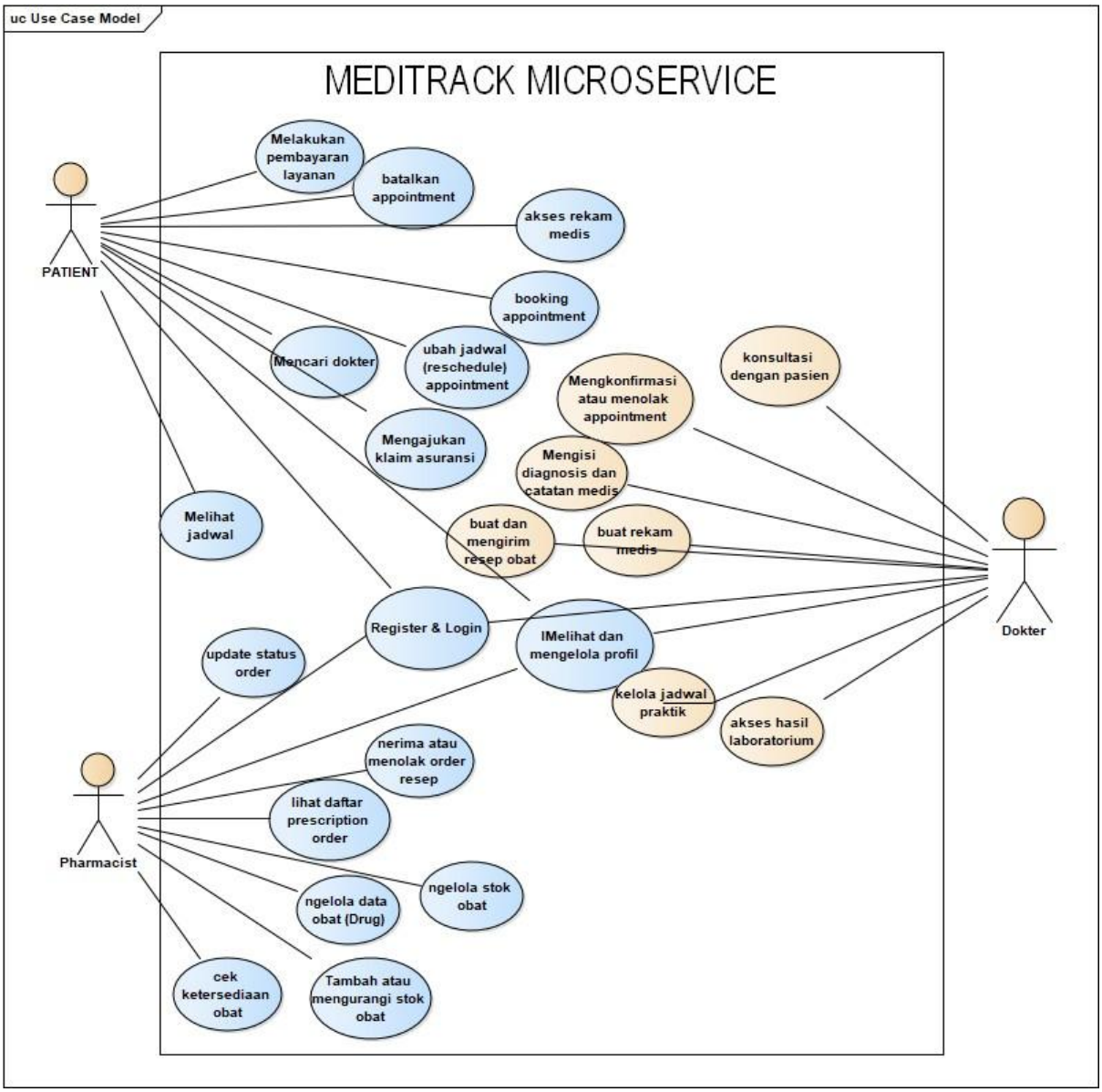
6. **[Bonus Task] Identify potential performance bottlenecks in your architecture and propose mitigation strategies!**

No	Bottleneck	Root Cause	Mitigation Strategy	Implementation	Priority
1	**Database Query**	Terlalu banyak query sekaligus dari modul berbeda	Add eager loading with with(), Query optimization	Appointment::with(['patient', 'doctor'])->get()	HIGH
2	**Database Query**	Missing database indexes	Add indexes on frequently searched columns	Create migration for indexes on email, phone, name	HIGH
3	**Database Connection**	Monolith scales horizontally,	Increase pool size, Implement	Configure max_connections in DB, Use PgBouncer	MEDIUM

		each instance needs connections	connection pooling		
3	**API Response**	Memory spike, Slow serialization	Add pagination with limit/offset	Modify controllers: ->paginate(15)	HIGH
4	**Authentication**	No token caching layer	Cache token in Redis, Blacklist management	Implement token caching middleware	MEDIUM
5	**Real-time Features**	Delayed notifications, Wasted requests	Implement WebSocket (Laravel Reverb), Server-sent events	Setup Laravel Reverb or Soketi	LOW

Case Study 2: MediTrack Transformation

1. **Soal ini dikerjakan pada branch case_study2**
2. **To simplify the main concept and understanding, please provide the use case based on the case study!**



1. **Patient (Pasien)**

Pasien merupakan pengguna utama yang menggunakan platform untuk mendapatkan layanan kesehatan secara digital.

- Registrasi akun sebagai pasien
- Login ke dalam sistem
- Melihat dan mengelola profil pribadi
- Mencari dokter berdasarkan spesialisasi atau lokasi
- Melihat jadwal ketersediaan dokter
- Melakukan booking appointment
- Mengubah jadwal (reschedule) appointment
- Membatalkan appointment
- Menerima notifikasi (appointment, resep, pembayaran)
- Melihat riwayat appointment
- Mengakses rekam medis (EHR)
- Melihat hasil laboratorium
- Melihat dan menebus resep obat
- Memilih pharmacy untuk pengambilan obat
- Melakukan pembayaran layanan
- Mengajukan klaim asuransi
- Melihat status pembayaran dan klaim
- Mengunduh invoice atau bukti pembayaran

2. Doctor (Dokter)

Dokter berperan dalam memberikan layanan medis dan mengelola data klinis pasien.

Daftar Use Case:

- Login ke sistem sebagai dokter
- Mengelola profil dokter (spesialisasi, jadwal, dll)
- Melihat daftar appointment pasien
- Mengkonfirmasi atau menolak appointment
- Melakukan konsultasi dengan pasien
- Membuat rekam medis (Medical Record)
- Mengisi diagnosis dan catatan medis
- Menginput hasil pemeriksaan

- Membuat dan mengirim resep obat (Prescription)
- Melihat riwayat medis pasien
- Mengakses hasil laboratorium pasien
- Mengelola jadwal praktik
- Melihat performa (analytics)
- Menerima notifikasi terkait appointment dan pasien

3. Pharmacist (Apoteker)

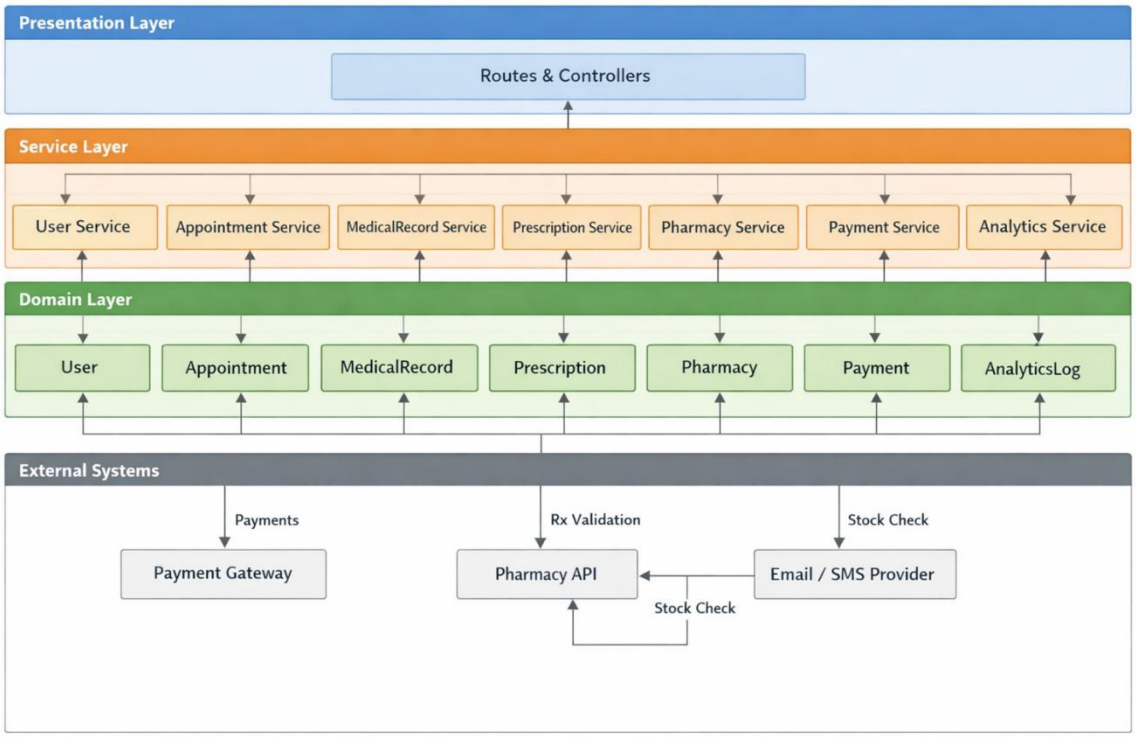
Apoteker bertanggung jawab dalam pengelolaan obat dan pemrosesan resep.

- Login ke sistem sebagai apoteker
- Mengelola profil pharmacy
- Melihat daftar prescription order
- Menerima atau menolak order resep
- Menyiapkan dan mendistribusikan obat
- Mengupdate status order (diproses, selesai)
- Mengelola stok obat (inventory)
- Menambah atau mengurangi stok obat
- Mengecek ketersediaan obat
- Melihat riwayat transaksi obat
- Mengelola data obat (Drug)
- Melihat laporan penggunaan obat
- Menerima notifikasi order baru

3. Architecture Diagram

a. Draw both the monolith and the microservices architecture.

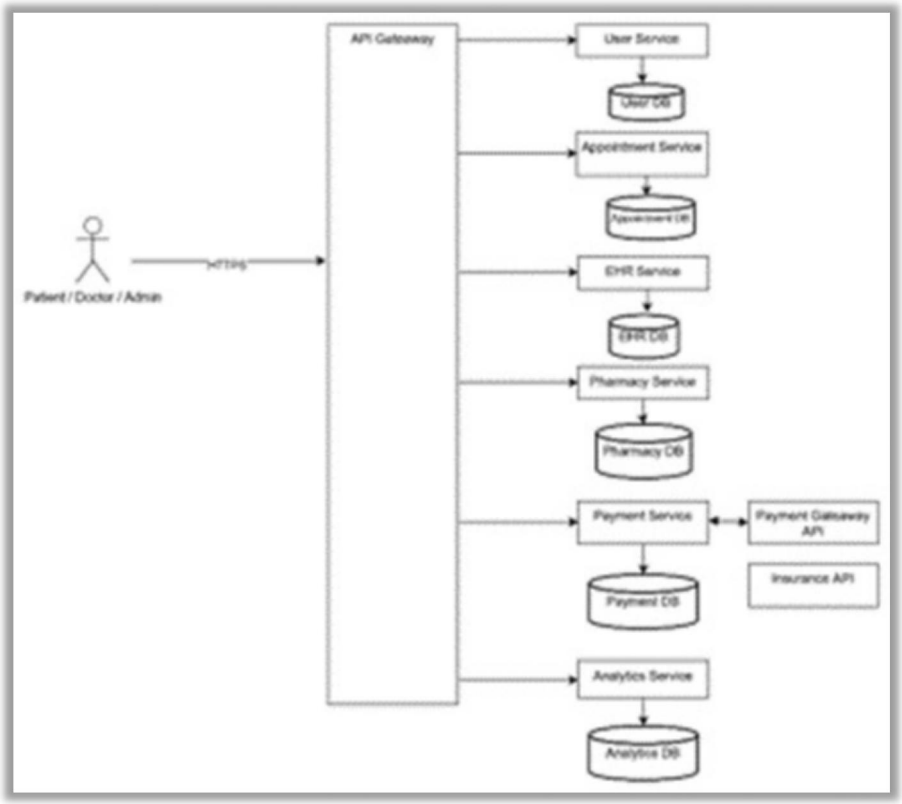
Sebelum (monolith):



Karakteristik:

- Semua modul berada dalam satu proses aplikasi.
- Komunikasi antar modul melalui pemanggilan metode langsung (tight coupling).
- Semua modul mengakses database yang sama (shared schema).
- Deployment satu kesatuan; perubahan kecil memerlukan redeploy seluruh aplikasi.

Setelah (microservice):



Karakteristik:

- Setiap service berdiri sendiri, memiliki basis kode dan database sendiri (polyglot persistence).
- Komunikasi antar service melalui REST API (sinkron) dan message queue (asinkron) untuk event.
- API Gateway sebagai pintu masuk tunggal, menangani routing, autentikasi, dan agregasi.
- Service discovery dapat diimplementasikan (DNS atau tools seperti Consul).
- Deployment independen: setiap service dapat di-deploy tanpa mempengaruhi yang lain.

Aliran Data:

1. Klien → API Gateway (port 8080).
2. Gateway meneruskan request ke service yang sesuai berdasarkan path (misal /api/v1/register → Auth Service).
3. Service memproses request, mengakses database-nya sendiri.
4. Jika perlu berkomunikasi dengan service lain (misal Appointment Service memvalidasi user), ia melakukan panggilan REST ke service tersebut.

5. Response dikembalikan ke klien melalui gateway.

Bagaimana Arsitektur Baru Mencakup Fungsionalitas Lama:

- Fungsionalitas lama tetap dipertahankan dengan memindahkannya secara bertahap ke microservices.
- Selama migrasi, monolit masih berjalan. API Gateway mengarahkan sebagian request ke microservices yang sudah siap, sisanya ke monolit (Strangler Pattern).
- Setelah semua service berjalan, monolit dimatikan. Tidak ada fungsionalitas yang dihapus, hanya dipindahkan.

4. Reasoning and Trade-offs

Aspek	Monolith	Microservices	Alasan Transformasi
Codebase	Single codebase untuk semua modul	Modul terpisah per service	Kemudahan maintenance, reduce complexity
Deployment	Semua naik-turun bersama	Deploy independen per service	Zero-downtime deploy, faster iteration
Scaling	Scale seluruh aplikasi	Scale per modul yang need	Cost efficient, targeted scaling
Database	1 database shared	DB per service (polyglot)	Data isolation, independence
Team	Semua tim berbagi codebase	Tim own their service	Autonomous team, faster development
Failure	1 bug = seluruh app down	1 service down ≠ down semua	Resilience, fault isolation
Tech Stack	Sama untuk semua (PHP/Laravel)	Berbeda per service	Best tech untuk setiap case

5. Migration Strategy

Tahapan:

Fase 1 Persiapan Infrastruktur

- Setup API Gateway (Nginx atau custom Go).
- Setup service registry (Consul/etcd) untuk discovery.
- Containerize semua service dengan Docker (opsional).

Fase 2 Migrasi Auth Service

- Pisahkan modul autentikasi dari monolit.
- API Gateway mengarahkan request /api/v1/register, /login ke Auth Service baru.
- Monolit tetap menangani modul lain. Auth Service memiliki database sendiri (auth_db).

Fase 3 Migrasi Appointment Service

- Pisahkan modul janji temu.
- Appointment Service membutuhkan validasi user; ia akan memanggil Auth Service via REST.
- Selama migrasi, data janji temu dapat ditulis ke dua tempat (dual write) untuk menjaga konsistensi.

Fase 4 Migrasi Service Inti Lainnya

- EHR Service, Pharmacy Service, Payment Service, Analytics Service dipisahkan secara bertahap.
- Untuk komunikasi antar service, kita gunakan REST API; untuk event yang tidak membutuhkan respons langsung (misal notifikasi), gunakan message queue (RabbitMQ/Kafka).

Fase 5 Penonaktifan Monolit

- Setelah semua fungsionalitas berjalan di microservices, monolit dimatikan.
- Semua traffic dialihkan ke microservices melalui API Gateway. Penting:
- Selama migrasi, kedua arsitektur berjalan paralel.
- API Gateway menggunakan routing berdasarkan versi API atau feature flag untuk menentukan ke mana request dikirim.
- Data yang tersebar di beberapa database disinkronkan melalui event sourcing atau CQRS jika diperlukan.

6. [Implementasi] Monolith Representation (Simulation)

a. Provide a simplified java code structure (classes, packages, modules) representing the monolithic architecture.

- DatabaseConnection

client.go U | go.mod M | appointment_handler.go M | DatabaseConnection.java U

Representasi Monolit java > src > main > java > com > meditrack > database > DatabaseConnection.java

```java
package com.meditrack.database;

import java.sql.*;

/**
 * SHARED DATABASE CONNECTION - Monolithic Pattern
 * Semua modul menggunakan koneksi database yang sama (single point of failure)
 * Tidak ada isolasi database per service
 */
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private static Connection connection;

    private DatabaseConnection() {
        // Singleton - koneksi tunggal untuk seluruh aplikasi
    }

    Trae: Explain | Doc |
    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
            initializeConnection();
        }
        return instance;
    }

    Trae: Explain | Doc |
    private static void initializeConnection() {
        try {
            // Koneksi ke database tunggal
            String url = "jdbc:mysql://localhost:3306/meditrack_monolith";
            String user = "root";
```

```java
    Trae: Explain | Doc |
    public Connection getConnection() {
        if (connection == null) {
            initializeConnection();
        }
        return connection;
    }

    Trae: Explain | Doc |
    public void executeUpdate(String sql, Object... params) throws SQLException {
        PreparedStatement stmt = connection.prepareStatement(sql);
        for (int i = 0; i < params.length; i++) {
            stmt.setObject(i + 1, params[i]);
        }
        stmt.executeUpdate();
        stmt.close();
    }

    Trae: Explain | Doc |
    public ResultSet executeQuery(String sql, Object... params) throws SQLException {
        PreparedStatement stmt = connection.prepareStatement(sql);
        for (int i = 0; i < params.length; i++) {
            stmt.setObject(i + 1, params[i]);
        }
        return stmt.executeQuery();
    }
}
```

- Models

Appointment

```
Representasi Monolit java > src > main > java > com > meditrack > models > Appointment.java
1   package com.meditrack.models;
2
3   import java.time.LocalDateTime;
4
5   /**
6    * SHARED APPOINTMENT MODEL
7    */
8   public class Appointment {
9       private int id;
10      private int patientId;
11      private int doctorId;
12      private LocalDateTime appointmentDate;
13      private String type;
14      private String location;
15      private int duration;
16      private String description;
17      private String status;
18      private LocalDateTime createdAt;
19
20      public Appointment() {}
21
22      public Appointment(int patientId, int doctorId, LocalDateTime appointmentDate) {
23          this.patientId = patientId;
24          this.doctorId = doctorId;
25          this.appointmentDate = appointmentDate;
26          this.status = "scheduled";
27          this.createdAt = LocalDateTime.now();
28      }
29
30      // Getters & Setters
        Trae: Explain | Doc | X
```

```
29
30      // Getters & Setters
        Trae: Explain | Doc | X
31      public int getId() { return id; }
        Trae: Explain | Doc | X
32      public void setId(int id) { this.id = id; }
33
        Trae: Explain | Doc | X
34      public int getPatientId() { return patientId; }
        Trae: Explain | Doc | X
35      public void setPatientId(int patientId) { this.patientId = patientId; }
36
        Trae: Explain | Doc | X
37      public int getDoctorId() { return doctorId; }
        Trae: Explain | Doc | X
38      public void setDoctorId(int doctorId) { this.doctorId = doctorId; }
39
        Trae: Explain | Doc | X
40      public LocalDateTime getAppointmentDate() { return appointmentDate; }
        Trae: Explain | Doc | X
41      public void setAppointmentDate(LocalDateTime appointmentDate) { this.appointmentDate
42
        Trae: Explain | Doc | X
43      public String getType() { return type; }
        Trae: Explain | Doc | X
44      public void setType(String type) { this.type = type; }
45
        Trae: Explain | Doc | X
46      public String getLocation() { return location; }
        Trae: Explain | Doc | X
47      public void setLocation(String location) { this.location = location; }
```

User

```
Representasi Monolit java > src > main > java > com > meditrack > models > User.java
1   package com.meditrack.models;
2
3   import java.time.LocalDateTime;
4
5   /**
6    * SHARED USER MODEL - Monolithic Pattern
7    * Semua modul menggunakan model yang sama (tight coupling pada struktur data)
8    * Perubahan pada model mempengaruhi semua modul
9    */
10  public class User {
11      private int id;
12      private String name;
13      private String email;
14      private String password;
15      private String phone;
16      private String address;
17      private String role; // doctor, patient, admin, pharmacist
18      private String status;
19      private String specialty; // Untuk dokter
20      private String licenseNumber; // Untuk dokter
21      private String insuranceProvider; // Untuk pasien
22      private LocalDateTime createdAt;
23      private LocalDateTime updatedAt;
24
25      // Constructors
26      public User() {}
27
28      public User(String name, String email, String phone, String role) {
29          this.name = name;
30          this.email = email;
31          this.phone = phone;
32          this.role = role;
```

```
client.go U   go.mod M   appointment_handler.go M   User.java U
Representasi Monolit java > src > main > java > com > meditrack > models > User.java
10  public class User {
62      public String getSpecialty() { return specialty; }
63      public void setSpecialty(String specialty) { this.specialty = specialty; }
64
65      public String getLicenseNumber() { return licenseNumber; }
66      public void setLicenseNumber(String licenseNumber) { this.licenseNumber = licenseNumb
67
68      public String getInsuranceProvider() { return insuranceProvider; }
69      public void setInsuranceProvider(String insuranceProvider) { this.insuranceProvider =
70
71      public LocalDateTime getCreatedAt() { return createdAt; }
72      public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
73
74      public LocalDateTime getUpdatedAt() { return updatedAt; }
75      public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
76
77      @Override
78      public String toString() {
79          return String.format("User{id=%d, name='%s', email='%s', role='%s'}", id, name, e
80      }
81  }
82
```

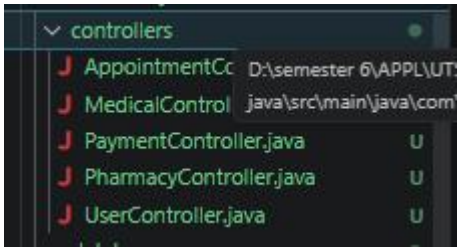
MedicalRecord

```java
package com.meditrack.models;

import java.time.LocalDateTime;

/**
 * SHARED MEDICAL RECORD MODEL
 */
public class MedicalRecord {
    private int id;
    private int patientId;
    private int doctorId;
    private String diagnosis;
    private String treatment;
    private String notes;
    private LocalDateTime recordDate;
    private LocalDateTime createdAt;

    public MedicalRecord() {}

    public MedicalRecord(int patientId, int doctorId, String diagnosis) {
        this.patientId = patientId;
        this.doctorId = doctorId;
        this.diagnosis = diagnosis;
        this.recordDate = LocalDateTime.now();
        this.createdAt = LocalDateTime.now();
    }

    // Getters & Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getPatientId() { return patientId; }
    public void setPatientId(int patientId) { this.patientId = patientId; }
```

```java
public class MedicalRecord {
    public int getPatientId() { return patientId; }
    public void setPatientId(int patientId) { this.patientId = patientId; }

    public int getDoctorId() { return doctorId; }
    public void setDoctorId(int doctorId) { this.doctorId = doctorId; }

    public String getDiagnosis() { return diagnosis; }
    public void setDiagnosis(String diagnosis) { this.diagnosis = diagnosis; }

    public String getTreatment() { return treatment; }
    public void setTreatment(String treatment) { this.treatment = treatment; }

    public String getNotes() { return notes; }
    public void setNotes(String notes) { this.notes = notes; }

    public LocalDateTime getRecordDate() { return recordDate; }
    public void setRecordDate(LocalDateTime recordDate) { this.recordDate = recordDate; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
}
```

- Sevice

- Controller



7. Microservices Mapping

1. Mapping Table

Modul Monolit	Microservice	Database	Port	Komunikasi
User Module	User Service	meditrack_users	3001	REST API
Appointment	Appointment Service	meditrack_appointments	3002	REST API
Medical/EHR	Medical Service	meditrack_medical	3003	REST API
Pharmacy	Pharmacy Service	meditrack_pharmacy	3004	REST API
Payment	Payment Service	meditrack_payment	3005	REST API
Analytics	Analytics Service	meditrack_analytics	3006	REST API